Canopy

Security Assessment

March 11th, 2025 — Prepared by OtterSec

Bartłomiej Wierzbiński dark@osec.io

Fineas Silaghi fedex@osec.io

Robert Chen r@osec.io

Table of Contents

Executive Summary **3**

Overview 3

Key Findings 3

Scope **4**

Findings **5**

Vulnerabilities **6**

OS-CPY-ADV-00 | Ambiguous Handling of Asset Contributions 8

OS-CPY-ADV-01 | Improper Transfer via Shared Resource Account 10

OS-CPY-ADV-02 | Incorrect Claim Calculations 11

OS-CPY-ADV-03 | Message Validation Bypass 13

OS-CPY-ADV-04 | Inconsistency in Emergency Withdrawal Procedure 14

OS-CPY-ADV-05 | Unrestricted Cross-Chain Messaging 15

OS-CPY-ADV-06 | Signature Replay in Claim Functionality 16

General Findings **17**

OS-CPY-SUG-00 | Pause State Transition Inconsistency 19

OS-CPY-SUG-01 | Ambiguous Dumped Amount Reporting 20

OS-CPY-SUG-02 | Unnecessary Boolean Checks in Claim Tracking 21

OS-CPY-SUG-03 | Code Optimization 22

OS-CPY-SUG-04 | Modifying Mapping Update Logic 23

OS-CPY-SUG-05 | Missing Validation Logic 24

OS-CPY-SUG-06 | Code Refactoring 25

OS-CPY-SUG-07 | Code Maturity 27

© 2025 Otter Audits LLC. All Rights Reserved.

OS-CPY-SUG-08 | Code Clarity 28

OS-CPY-SUG-09 | Documentation Enhancement 30

OS-CPY-SUG-10 | Code Redundancy 32

OS-CPY-SUG-11 | Unutilized Code 35

Appendices

Vulnerability Rating Scale **37**

Procedure **38**

© 2025 Otter Audits LLC. All Rights Reserved.

01 — Executive Summary

Overview

Canopy engaged OtterSec to assess the `cornucopia-move` program. This assessment was conducted between January 10th and March 10th, 2025. We also conducted a follow-up audit from April 18th to April 19th, 2025. For more information on our auditing methodology, refer to Appendix B.

Key Findings

We produced 19 findings throughout this audit engagement.

In particular, we identified several critical vulnerabilities, including one concerning the protocol's ambiguous handling of asset contributions based on the resource account's balances, which may result in incorrect asset processing, creating a denial of service or loss of funds due to misinterpretation of incoming contributions (OS-CPY-ADV-01), and a lack of validation for incoming messages, allowing attackers to exploit griefing and funds-stealing vulnerabilities by sending malformed or empty messages to misappropriate or dump funds (OS-CPY-ADV-03).

Additionally, the creator of the single asset base deployer may directly withdraw funds from the primary fungible store via the FA API, bypassing the emergency withdrawal restrictions (OS-CPY-ADV-05).

We also made recommendations regarding modifications to the codebase for improved functionality, efficiency, and robustness (OS-CPY-SUG-06) and suggested removing redundant and unutilized code instances (OS-CPY-SUG-10, OS-CPY-SUG-11). Moreover, we advised adhering to coding best practices (OS-CPY-SUG-07) and including additional safety checks within the codebase to improve security (OS-CPY-SUG-05).

Furthermore, we highlighted certain modifications to rectify informational and typographical errors in the comments to implement proper error handling and emit events to log state changes for improved transparency and debugging (OS-CPY-SUG-08).

© 2025 Otter Audits LLC. All Rights Reserved.

02 — Scope

The source code was delivered to us in a Git repository at https://github.com/Canopyxyz/cornucopia-move. This audit was performed against commit fcca73b. We further reviewed PR#30.

A brief description of the program is as follows:

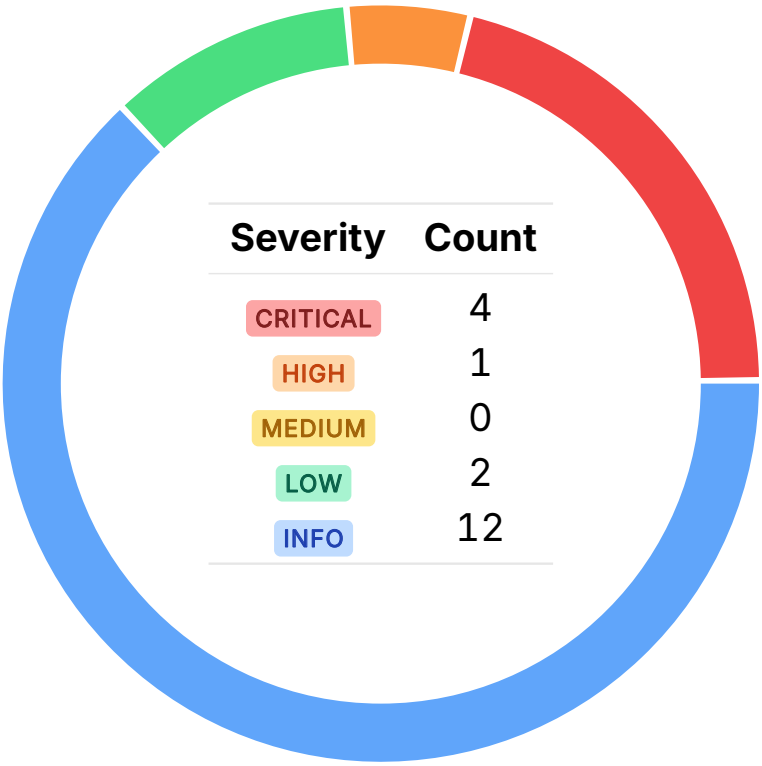
Name	Description
cornucopia-move	A cross-chain liquidity deployment and rewards management system that bridges assets from Ethereum to the Move blockchain using LayerZero. It manages user deposits, investments in underlying protocols, and reward distributions.

© 2025 Otter Audits LLC. All Rights Reserved.

03 — Findings

Overall, we reported 19 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



Severity	Count
CRITICAL	4
HIGH	1
MEDIUM	0
LOW	2
INFO	12

© 2025 Otter Audits LLC. All Rights Reserved.

04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in Appendix A.

ID	Severity	Status	Description
OS-CPY-ADV-00	CRITICAL	RESOLVED	The protocol's ambiguous handling of asset contributions based on the resource account's balances may result in incorrect asset processing, creating a denial of service or a loss of funds due to misinterpretation of incoming contributions.
OS-CPY-ADV-01	CRITICAL	RESOLVED	`notify_funds` transfers all `base_asset` from the global resource account to the deployer's primary fungible store. This approach may result in interference among multiple `SingleAssetBaseDeployer` objects sharing the same asset.
OS-CPY-ADV-02	CRITICAL	RESOLVED	In `process_claim`, the claim calculations during the lockup phase fail to account for withdrawals and deployments, resulting in incorrect share and undeployed base asset distribution.
OS-CPY-ADV-03	CRITICAL	RESOLVED	There is a lack of validation for incoming messages in `handle_batch_contribution_message`, allowing attackers to exploit griefing and funds-stealing vulnerabilities by sending malformed or empty messages to misappropriate or dump funds.
OS-CPY-ADV-04	HIGH	RESOLVED	The creator of the `SingleAssetBaseDeployer` may directly withdraw funds from the primary fungible store via the fungible asset API, bypassing the emergency withdrawal restrictions.
OS-CPY-ADV-05	LOW	RESOLVED	`example_message_sender` allows anyone to send arbitrary cross-chain messages, resulting in unauthorized interactions or exploits.

© 2025 Otter Audits LLC. All Rights Reserved.

OS-CPY-ADV-06	LOW	RESOLVED	`claim_relevant_assets` is vulnerable to replay attacks, allowing the possibility of retransmitting valid claim requests within the deadline window.

© 2025 Otter Audits LLC. All Rights Reserved.

Ambiguous Handling of Asset Contributions CRITICAL OS-CPY-ADV-00

Description

The vulnerability in the dual asset module relates to the ambiguity in deciding which asset contribution to process based on the primary fungible store balances of the resource account. The contributions in dual asset are made for one asset at a time. Thus, the protocol must make two separate contributions - for both assets. The protocol chooses which asset to proceed with based on the balance of assets in the primary fungible store of the resource account. This ambiguity may result in denial of service or loss of funds scenarios.

This may be exploited in two ways as illustrated below:

Proof of Concept

Below scenario takes into account the presence of an attacker:

1. A legitimate user contributes `asset_1` to the pool. The fungible assets for `asset_1` are minted to the resource account, and the protocol waits for the compose trigger.
2. An attacker deposits `asset_0` into the primary fungible store of the resource account, ensuring that the deposit is equal to the amount of the incoming `asset_1` contribution.
3. The `asset_1` deposit is processed, and the protocol assumes it is an `asset_0` contribution because it has enough of `asset_0` to continue. Thus, this contribution is processed further as `asset_0`, which may be worth less than `asset_1`.

The following example illustrates the case where there is no attacker:

1. `asset_0` and `asset_1` contributions are submitted by different users. Fungible assets for both assets are posted to the resource account, and the protocol waits for the `compose` trigger.
2. The protocol receives the `compose` message for `asset_1` and checks asset balances. If the balance of `asset_0` is equal to or greater than the `asset_1` contribution, the protocol will incorrectly process the contribution as `asset_0`. If `asset_0` < `asset_1`, nothing happens.
3. The protocol then processes the `asset_0` contribution. If the previous contribution was mistakenly processed as `asset_0`, the protocol will try to process this as `asset_1`, given that enough `asset_1` exists. If not enough `asset_1` exists, it will be discarded.

Remediation

The protocol should process the assets based on the message source, not the asset balances.

© 2025 Otter Audits LLC. All Rights Reserved.

Patch

Resolved in 5abad86 and 6bac9d0.

© 2025 Otter Audits LLC. All Rights Reserved.

Improper Transfer via Shared Resource Account CRITICAL OS-CPY-ADV-01

Description

`notify_funds` in `single_asset_base_deployer` processes LayerZero compose messages containing information about bridged assets. It records individual contributor amounts for funds that were directly bridged. However, the current implementation transfers all available `base_asset` from the global resource account to the deployer's primary fungible store without distinguishing between different deployers. Multiple `SingleAssetBaseDeployer` objects may share the same `base_asset`.

>_ *deployers/single-asset/sources/base/single_asset_base_deployer.move* RUST

```rust
public(friend) fun notify_funds(
    base_deployer_obj_address_bytes: vector<u8>,
    eth_contributors: vector<vector<u8>>,
    contribution_amounts: vector<u64>
): (u64, u64) acquires GlobalConfig, SingleAssetBaseDeployer {
    [...]
    // Transfer any asset from resource account to deployer
    let resource_balance = primary_fungible_store::balance(resource_account_addr, base_asset);
    if (resource_balance > 0) {
        let funds = primary_fungible_store::withdraw(&resource_signer, base_asset,
            ↪  resource_balance);
        primary_fungible_store::deposit(deployer_addr, funds);
    };
    [...]
}
```

Transferring all available assets from the global resource account to the deployer's primary fungible store without proper segregation may result in assets intended for one deployer to be allocated to another. Additionally, if one deployer consumes all available assets, others may be unable to function properly.

Remediation

Ensure that funds designated for specific `SingleAssetBaseDeployer` are separated.

Patch

Resolved in 5abad86.

© 2025 Otter Audits LLC. All Rights Reserved.

Incorrect Claim Calculations CRITICAL OS-CPY-ADV-02

Description

`process_claim` in `SingleAssetBaseDeployer` contains a flaw in how it calculates the undeployed base assets and shares when contributors claim their funds. Specifically, when a claim occurs during the lockup phase and is followed by another deployment, the subsequent claims may become inconsistent, resulting in incorrect share allocations and undeployed base distributions. The function does not correctly account for changes in total shares and base assets caused by partial claims and subsequent deployments.

Proof of Concept

1. There are 10 contributors, each contributing 1000 base assets. The `total_base_received` is 10000, and `last_base_balance` is 10000.
2. When the first deployment occurs, 8000 base assets are deployed to receive 8000 shares. Now `total_base_deployed` is 8000, `total_shares_received` is 8000, and `last_base_balance` is 2000.
3. One contributor claims their funds during the lockup phase. They claim 200 undeployed base assets and 800 shares. `last_base_balance` is now 1800, and while `total_shares_received` is still 8000, the primary fungible store actually holds only 7200 because 800 shares were withdrawn.
4. A second deployment occurs, and another 1000 base assets are deployed to exchange 1000 more shares. `last_base_balance` is now 800, and `total_shares_received` is updated to 8200. The function updates `total_shares_received` to equal the balance of the primary fungible store for share assets.
5. After the lockup phase ends, the remaining 9 contributors try to claim their funds. The undeployed base asset calculation distributes 100 base per contributor (`1000*1000/10000`), totaling 900, which the protocol does not have. The shares calculation distributes 820 shares per contributor (`1000*8200/10000`).
6. Thus, it will not be possible for the last claimant to claim their 100 base assets since only 800 remain. Also, the primary fungible store is left with 1640 unclaimed shares, breaking the expected fair distribution of funds.

Remediation

Ensure that the formula utilized to calculate shares and undeployed amounts accounts for changes resulting from withdrawals and deployments, reflecting these changes in the total shares and base asset.

© 2025 Otter Audits LLC. All Rights Reserved.

Patch

Resolved in e461d18.

© 2025 Otter Audits LLC. All Rights Reserved.

Message Validation Bypass CRITICAL OS-CPY-ADV-03

Description

`handle_batch_contribution_message` fails to validate the sender of the composed message, which opens the door to griefing and funds-stealing attacks. An attacker may deploy a `SingleAssetBaseDeployer` with a vault that has a `base_asset` that matches the victim's `SingleAssetBaseDeployer`. An attacker would then pause the `SingleAssetBaseDeployer` and send a malformed compose message. All fungible assets from the `resource_account_addr` store with this metadata would be moved to the `timelocked_asset_dump` address.

>_ *deployers/single-asset/sources/base/single_asset_base_deployer.move* RUST

```rust
public(friend) fun notify_funds(
    base_deployer_obj_address_bytes: vector<u8>,
    eth_contributors: vector<vector<u8>>,
    contribution_amounts: vector<u64>
): (u64, u64) acquires GlobalConfig, SingleAssetBaseDeployer {
    [...]
    if (vector::length(&eth_contributors) != vector::length(&contribution_amounts)) {
        dump_unnotified_balances(deployer_ref, global_config, unnotified_balance);
        return (TRACE_MISMATCHED_VECTORS, 0)
    };
    [...]
}
```

Furthermore, an attacker may exploit unprocessed contributions (such as when bridged fungible assets are minted to the store, but the original contribution message has not been processed yet) to steal funds. The attacker may send a `notify_funds` message to claim these unprocessed assets. Later, when `notify_funds` checks if total contributions exceed the unnotified balance, the original contribution message would be discarded.

Remediation

Implement proper validation of incoming messages. Prevent unauthorized `notify_funds` calls from succeeding without the necessary checks.

Patch

Resolved in 03bb14d.

© 2025 Otter Audits LLC. All Rights Reserved.

Inconsistency in Emergency Withdrawal Procedure HIGH OS-CPY-ADV-04

Description

The protocol's emergency withdraw procedure may be initiated and completed without the ability for the global protocol administrator to stop it. Specifically, the `SingleAssetBaseDeployer` owner has the ability to bypass protocol restrictions, including emergency withdrawals. Due to the way in which the primary fungible store ownership is checked, it is possible for the `SingleAssetBaseDeployer` owner to directly interact with its primary fungible store via the fungible asset API.

Remediation

Ensure that the ownership of the `SingleAssetBaseDeployer` object is delegated to avoid abuse from the creator.

Patch

Resolved in 8b29ad3.

© 2025 Otter Audits LLC. All Rights Reserved.

Unrestricted Cross-Chain Messaging LOW OS-CPY-ADV-05

Description

`oapp::example_message_sender` allows anyone to send arbitrary cross-chain messages to the OApp's configured peers without any access control or authorization checks. Since it does not verify the caller's authorization, any user may send messages from this OApp. The impact of this issue depends on who receives the message and how the message contents are processed on the receiving end.

Remediation

Remove this feature or have its access restricted.

Patch

Resolved in 5549c0f.

© 2025 Otter Audits LLC. All Rights Reserved.

Signature Replay in Claim Functionality LOW OS-CPY-ADV-06

Description

`claim_relevant_assets` is susceptible to replay attacks within the deadline window. If a user deposits, withdraws before the lockup phase, and then deposits again, an attacker may replay the `claim` call to withdraw funds on behalf of the user during the deadline window. Moreover, if the phase transitions to the lockup phase in the meantime, the user will incur additional penalties for withdrawing during lockup.

Remediation

Restrict the use of the signature-style claim and document it clearly, or remove it entirely. Currently, high-level overview documentation only references the address mapping claim style.

Patch

Resolved in b08b177.

© 2025 Otter Audits LLC. All Rights Reserved.

05 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-CPY-SUG-00	A pause state change may be initiated before deployment and completed afterward, potentially creating inconsistencies, while other timelocked changes will not be executed or canceled if conditions are not met.
OS-CPY-SUG-01	The `dumped_amount` field in the `MessageProcessingEvent` is utilized inconsistently, as it refers to different asset types in various parts of the `handle_batch_contribution_message`.
OS-CPY-SUG-02	The utilization of boolean values in the claimed lockup tables is unnecessary, as checking the existence of the address is sufficient.
OS-CPY-SUG-03	The overall code may be streamlined further to reduce complexity and improve efficiency.
OS-CPY-SUG-04	The current mapping update logic in `configure_move_claim_address` may be simplified by utilizing `table::upsert`, which handles both the insertion and updating of key-value pairs.
OS-CPY-SUG-05	There are several instances where proper validation is not performed, resulting in potential issues.
OS-CPY-SUG-06	Recommendation for modifying the codebase for improved efficiency, functionality, and robustness.
OS-CPY-SUG-07	Suggestions regarding inconsistencies in the codebase and ensuring adherence to coding best practices.

© 2025 Otter Audits LLC. All Rights Reserved.

OS-CPY-SUG-08	Modifications to the codebase to improve transparency through proper event emission and error handling.
OS-CPY-SUG-09	The natspec comments and documentation may be updated to accurately reflect the current functionality and ensure consistency.
OS-CPY-SUG-10	There are multiple instances of redundant code that should be removed for better maintainability and clarity.
OS-CPY-SUG-11	The codebase contains multiple cases of unnecessary or unutilized code that should be removed.

© 2025 Otter Audits LLC. All Rights Reserved.

Pause State Transition Inconsistency OS-CPY-SUG-00

Description

`manage_timelocked_pause_state` allows an admin to initiate a change in the pause state before deployment, but this change may be finalized after the deployment. The `option::is_some(&new_pause_state)` check ensures that if a new pause state is provided, the function proceeds with the update only during the first phase. However, in order to execute the change, `option` must be none. This means that the `first_deployment_timestamp == 0` assert is only checked during change initiation and not at execution.

>_ deployers/single-asset/sources/base/single_asset_base_deployer.move RUST

```rust
public entry fun manage_timelocked_pause_state([...]) acquires SingleAssetBaseDeployer {
    let deployer_ref =
        ↪  borrow_global_mut<SingleAssetBaseDeployer(object::object_address(&deployer_obj));
    assert_signer_is_deployer_owner(deployer_obj, account);
    if (option::is_some(&new_pause_state)) {
        assert!(
            deployer_ref.first_deployment_timestamp == 0,
            E_CANNOT_MODIFY_PAUSE_AFTER_DEPLOYMENT
        );
    };
    timelocked::manage_timelocked_update(&mut deployer_ref.timelocked_pause,
        ↪  new_pause_state,new_delay);
}
```

After the deployment has taken place, the `first_deployment_timestamp` is set, preventing initialization of any future changes to the pause state, which effectively locks it. This is an admin-controlled function.

In other functions that manage timelocked values, if a change request is made but the change conditions are no longer fulfillable, the system does not allow the change to be executed or canceled.

Remediation

Enforce stricter checks to ensure that any pause state change may only be initiated and completed within a valid pre-deployment phase.

© 2025 Otter Audits LLC. All Rights Reserved.

Ambiguous Dumped Amount Reporting OS-CPY-SUG-01

Description

The issue pertains to the ambiguity in the `MessageProcessingEvent`'s `dumped_amount` field, particularly within `oapp::handle_batch_contribution_message`. In the first two calls to the event emission, the `dumped_amount` represents the amount of native fungible asset that is rejected or deposited into a dump account. However, in the last call (after successfully notifying the deployer), the `dumped_amount` is not necessarily the native fungible asset anymore. Instead, it is the amount of `base_asset` fungible assets. This creates ambiguity, as the same field is utilized to record different types of asset dumps.

Remediation

Introduce a separate field for the dumped native FA amount and another for the dumped `base_asset` fungible asset amount, as implemented in the dual asset module.

© 2025 Otter Audits LLC. All Rights Reserved.

Unnecessary Boolean Checks in Claim Tracking OS-CPY-SUG-02

Description

The `claimed_during_lockup` and `claimed_post_lockup` tables track whether an Ethereum address has claimed assets in specific phases. These tables store only the `true` value, indicating that an address has claimed rewards during each respective phase.

>_ *deployers/single-asset/sources/base/single_asset_base_deployer.move* RUST

```rust
struct SingleAssetBaseDeployer has key {
    [...]
    /// Maps ethereum addresses to whether they claimed during lockup phase, disqualifying them
        ↪ from MOVE rewards
    claimed_during_lockup: Table<vector<u8>, bool>,
    /// Mapping of ethereum address and whether the user has claimed post lockup and hence is
        ↪ unable to call claim_relevant_assets again
    claimed_post_lockup: Table<vector<u8>, bool>,
    [...]
}
```

Although the code frequently checks the value of these entries to determine if an address has claimed assets, the tables only store `true` value. As a result, the actual value is irrelevant, and the mere existence of an entry is sufficient to confirm that the address has claimed rewards during the corresponding phase. Therefore, checking the value of these entries is unnecessary.

Remediation

Check only whether `claimed_during_lockup[address]` exists, instead of checking both its existence and value.

© 2025 Otter Audits LLC. All Rights Reserved.

Code Optimization

OS-CPY-SUG-03

Description

1. In `single_asset_base_deployer::get_contribution`, `table::contains` is utilized to check if the `eth_address` is present in the contributions table. If present, the contribution amount is fetched via `table::borrow` and dereferenced, otherwise, it returns zero, indicating no recorded contribution. Replace the `table::contains` and `table::borrow` calls with `table::borrow_with_default` which checks for the key's existence and returns a default value if it does not, eliminating the need for an explicit if-else statement.

   >_ *deployers/single-asset/sources/base/single_asset_base_deployer.move* RUST

   ```rust
   public fun get_contribution(deployer_obj: Object<SingleAssetBaseDeployer>,
       ↪  eth_address:vector<u8>): u64 acquires SingleAssetBaseDeployer {
       let deployer_addr = object::object_address(&deployer_obj);
       let deployer_ref = borrow_global<SingleAssetBaseDeployer>(deployer_addr);
       if (table::contains(&deployer_ref.contributions, eth_address)) {
           *table::borrow(&deployer_ref.contributions, eth_address)
       } else { 0 }
   }
   ```

2. `message_decoding` may utilize `vector::slice` instead of extracting amounts individually in a loop. Additionally, there are redundant vector length checks during decoding that may be optimized.
3. `execute_oft_package_update` and `execute_eth_sender_update` utilize `table::upsert`, whereas `add` will be more optimal since the values are never swapped.
4. There are multiple instances of `primary_fungible_store::withdraw` followed by `primary_fungible_store::deposit` in the codebase, which essentially performs a withdrawal from one store and a deposit into another. Instead of utilizing separate `withdraw` and `deposit` calls, a direct `transfer` call will simplify the logic.

Remediation

Update the codebase with the above-listed optimizations.

© 2025 Otter Audits LLC. All Rights Reserved.

Modifying Mapping Update Logic

OS-CPY-SUG-04

Description

In `configure_move_claim_address`, the global mapping in the `GlobalConfig` is updated for associating an Ethereum address with a corresponding Movement address. The code that checks if the mapping exists (`table::contains`), and consequently updates or adds a new entry in the table based on whether the mapping exists or not (`table::borrow_mut` or `table::add`), should be refactored into `table::upsert` to simplify the code.

>_ *deployers/single-asset/sources/base/single_asset_base_deployer.move* RUST

```rust
public(friend) fun configure_move_claim_address(eth_address: vector<u8>, move_claim_address:
    ↪  vector<u8>) acquires GlobalConfig {
    [...]
    // Try to update the mapping safely
    if (table::contains(&global_config.claim_addresses, eth_address)) {
        let stored_move_addr = table::borrow_mut(&mut
            ↪  global_config.claim_addresses,eth_address);
        *stored_move_addr = move_claim_address;
    } else {
        table::add(&mut global_config.claim_addresses, eth_address, move_claim_address);
    };
}
```

Remediation

Refactor such constructions into `table::upsert` to handle both the existence check and the update/add operation in a single function call.

© 2025 Otter Audits LLC. All Rights Reserved.

Missing Validation Logic

OS-CPY-SUG-05

Description

1. Before creating a new emergency withdrawal proposal, `propose_emergency_withdraw` should verify whether there is already a pending proposal, and `clear_emergency_withdraw_proposal` should check if a proposal actually exists before attempting to clear the emergency withdrawal proposal.
2. `safe_decode_type_2_message` should validate that the message length is at least $204 + 32 + 32$ bytes, not just 204 bytes. This ensures the minimum size is met, even if there are no contributors or amounts. Sizes must still be included in the message, even when set to 0.
3. Currently, while maximum limits, such as `MAX_LOCK_DURATION`, are defined, there are no minimum limits. It is recommended to add restrictions for minimum values to ensure validation across the entire acceptable range.
4. `execute_emergency_withdraw` does not enforce the signer to be the owner, whereas `execute_sweep_resource_account` does. This could be unified to maintain consistency.
5. Claims should only be allowed after the corresponding fungible assets have been transferred. If contributions are notified and claims are enabled before the deployer's fungible asset store receives the funds, users may encounter errors, receive zero share and base assets, and lose their contribution entry.

Remediation

Add the validations stated above.

© 2025 Otter Audits LLC. All Rights Reserved.

Code Refactoring OS-CPY-SUG-06

Description

1. Utilize an already retrieved signer address during an ownership check in `single_asset_base_deployer::create`.
2. The `TRACE_DEPLOYER_PAUSED` and `TRACE_MISMATCHED_VECTORS` trace codes in `dual_asset_base_deployer` share the same value, which may result in confusion while debugging issues. Ensure that trace codes are distinct.

   >_ *deployers/dual-asset/sources/base/dual_asset_base_deployer.move* RUST

   ```rust
   module cornucopia_dual_deployers::dual_asset_base_deployer {
       [...]
       /// Deployer is paused
       const TRACE_DEPLOYER_PAUSED: u64 = 203;
       /// Vector lengths mismatched
       const TRACE_MISMATCHED_VECTORS: u64 = 203;
   }
   ```

3. In `initiate_deploy_funds` within `dual_asset_base_deployer`, `total_asset_X_deployed` should be updated with the actual deployed amounts derived from balance deltas, which may differ from the function's input parameters. Currently, this is only a suggestion, as the field is not utilized in any calculations.
4. In the existing implementation of `assert_valid_lock_duration`, the assertion fails if the `lock_duration` is equal to `MAX_LOCK_DURATION`. Modify the check to utilize `<=` instead of `<`.

   >_ *deployers/dual-asset/sources/base/dual_asset_base_deployer.move* RUST

   ```rust
   fun assert_valid_lock_duration(lock_duration: u64) {
       assert!(lock_duration < MAX_LOCK_DURATION, E_EXCEEDS_MAX_LOCK_DURATION);
   }
   ```

5. `finalize_deploy_funds` changed the comparison from `>` to `>=` when checking if `current_shares_balance >= pre_deploy_shares_balance`. This slippage check is critical and should fail if no new shares are obtained during a deployment. The check may be refactored to account for fake deployments while preserving its integrity in real deployments.

© 2025 Otter Audits LLC. All Rights Reserved.

6. Some project features, such as `batch_get_funds_tracking`, will no longer function correctly in `v1.1` due to changes in the notify and deploy flow. For example, fields such as `total_shares_received` or `total_base_deployed` may now return zero. These tracking fields should be updated to preserve their role in maintaining protocol visibility.

Remediation

Incorporate the above refactors.

© 2025 Otter Audits LLC. All Rights Reserved.

Code Maturity

OS-CPY-SUG-07

Description

1. The existing implementation of `safe_decode_type_2_message` in `cornucopia_std::message_decoding` returns a boolean `success` flag to indicate whether the decoding was successful, which provides limited insight into the nature of any failure. Instead, returning specific error codes will significantly improve the error handling and debugging process.

   >_ sources/libs/message_decoding.move RUST

   ```rust
   /// @param message - The raw bytes of the LayerZero message payload
   /// @return (bool, vector<u8>, vector<vector<u8>>, vector<u64>) - Tuple containing:
   ///   - success: whether it was able to successfully decode the message in the desired
   ↪  format
   ///   - base_deployer_bytes: 32-byte representation of deployer object address
   ///   - eth_contributors: Vector of 20-byte Ethereum addresses
   ///   - contribution_amounts: Vector of contribution amounts corresponding to addresses
   public fun safe_decode_type_2_message(message: &vector<u8>): (bool, vector<u8>,
   ↪  vector<vector<u8>>, vector<u64>) {
   [...]
   }
   ```

2. `notify_funds` expects `eth_contributors` and `contribution_amounts` to have equal lengths and be shorter than `MAX_CONTRIBUTORS_LENGTH`. While these checks were implicitly handled via message decoding in the previous `notify` flow, `permissioned_notify_funds` lacks such decoding and should explicitly assert these conditions to prevent potential errors.
3. In `single_asset_base_deployer::notify_funds` when an error condition is encountered due to paused states or mismatched vectors, the function returns zero as the second value (the dumped amount) instead of correctly returning the actual `unnotified_balance`.
4. The resource account sweep should include a defined mechanism to cancel the sweep proposal if needed.

Remediation

Implement the above-mentioned suggestions.

© 2025 Otter Audits LLC. All Rights Reserved.

Code Clarity

OS-CPY-SUG-08

Description

1. `satay_deployer::create_internal` is marked as public, even though it is intended to be called internally by `create`.

>_ deployers/single-asset/sources/concrete/satay_deployer.move RUST

```rust
public fun create_internal(creator: &signer, vault_address: address):
    ↪  Object<SingleAssetBaseDeployer> {
        [...]
}
```

2. In `get_deployer_from_32_bytes`, the parameter name `eth_bytes` suggests that it represents an Ethereum address, which is conventionally 20 bytes long. However, in this function, the parameter actually holds a 32-byte deployer address. To improve clarity, change the parameter name to clearly reflect its purpose.

>_ deployers/single-asset/sources/base/single_asset_base_deployer.move RUST

```rust
/// Looks up a deployer object by its 32-byte address
/// @return Option<Object<SingleAssetBaseDeployer>> - The deployer object if found
public fun get_deployer_from_32_bytes(eth_bytes: vector<u8>):
    ↪  Option<Object<SingleAssetBaseDeployer>> acquires GlobalConfig {
    let global_config = borrow_global<GlobalConfig>(@cornucopia_single_deployers);
    if (table::contains(&global_config.deployer_map, eth_bytes)) {
        option::some(*table::borrow(&global_config.deployer_map, eth_bytes))
    } else {
        option::none()
    }
}
```

3. The way `configure_move_claim_address` and `configure_move_claim_address_for_deployer` are structured renders successful execution indistinguishable from failure, as they silently return when specific conditions are not met. This may result in unnoticed failures and unexpected behaviors. Explicit error returns should be implemented to enhance transparency and facilitate debugging.

© 2025 Otter Audits LLC. All Rights Reserved.

4. Timelocked events such as `schedule_value_update`, `schedule_delay_update`, `execute_update`, and `cancel_update` should emit an event to ensure proper logging of state changes. Alos, `configure_move_claim_address` and `configure_move_claim_address_for_deployer` do not emit events to indicate state changes. To ensure proper logging of global state changes, all significant state-changing operations should emit events.

Remediation

Ensure proper error handling and emit events to log state changes for improved transparency and facilitate debugging.

© 2025 Otter Audits LLC. All Rights Reserved.

Documentation Enhancement OS-CPY-SUG-09

Description

1. The natspec for `propose_emergency_withdraw` states that it *"Creates or overwrites existing proposal"*, however, the function has no ability to overwrite an existing proposal. Similarly, for `claim_relevant_assets_with_move_address`, the comment claims that *"This is the second claim method that allows a Movement address that was configured via LayerZero"*, but there is no first method anymore. Rectify the natspec to reflect the correct information.

   >_ deployers/single-asset/sources/concrete/satay_deployer.move RUST

   ```rust
   /// Proposes an emergency withdrawal of funds
   /// @param owner - Signer proving caller is the owner
   /// @param deployer_obj - The deployer object
   /// @param asset - Asset to withdraw
   /// @param recipient - Address to receive the withdrawn funds
   /// @param amount - Amount to withdraw (None means withdraw all available)
   ///
   /// @notice Creates or overwrites existing proposal
   public entry fun propose_emergency_withdraw([...]) acquires SingleAssetBaseDeployer
       ↪ {[...]}
   ```

2. The comment for `single_asset_base_deployer::notify_funds` should document the newly introduced `skip_balance_check` parameter, including its purpose. Additionally, a comment explaining the rationale behind adding `permissioned_notify_funds` will improve code clarity and maintainability.

   >_ deployers/single-asset/sources/base/single_asset_base_deployer.move RUST

   ```rust
   /// @param base_deployer_obj_address_bytes - 32-byte representation of deployer object
       ↪ address
   /// @param eth_contributors - Vector of contributor Ethereum addresses as vector<u8>
   /// @param contribution_amounts - Vector of contribution amounts corresponding to
       ↪ eth_addresses
   [...]
   /// @dev The message payload is encoded on Ethereum using solidity abi.encode with format:
   /// (bytes32 base_deployer_obj_address, address[] eth_contributors, uint64[]
       ↪ contribution_amounts)
   public(friend) fun notify_funds(
       base_deployer_obj_address_bytes: vector<u8>,
       eth_contributors: vector<vector<u8>>,
       contribution_amounts: vector<u64>,
       skip_balance_check: bool
   ): u64 acquires GlobalConfig, SingleAssetBaseDeployer {[...]}
   ```

© 2025 Otter Audits LLC. All Rights Reserved.

3. PR#30 does not include the updated default timelock durations outlined in the scoping document. These changes should be implemented to ensure integrity between the codebase and the documentation.

Remediation

Update the `natspec` comments to accurately reflect the functionality and update the documentation to ensure consistency between the codebase.

© 2025 Otter Audits LLC. All Rights Reserved.

Code Redundancy

OS-CPY-SUG-10

Description

1. Across the codebase, there are repeated assertion checks to verify whether the deployment has started (`first_deployment_timestamp != 0`) and whether the lock duration has passed. Such repeated checks clutter the code and thus, extracting this logic into a separate helper function will reduce code duplication.

>_ deployers/single-asset/sources/base/single_asset_base_deployer.move RUST

```rust
public entry fun manage_timelocked_lock_duration([...]) acquires SingleAssetBaseDeployer {
    [...]
    if (deployer_ref.first_deployment_timestamp != 0) { // only check if deployment has
        ↪ started
        assert!(
            current_time < deployer_ref.first_deployment_timestamp +
                ↪ timelocked::get_value(&deployer_ref.timelocked_lock_duration),
            E_LOCK_DURATION_PASSED
        );
    };
    [...]
}
```

2. Currently, `message_decoding::safe_decode_type_2_message` checks the message length twice for different values, once to check if it is less than 204 and again to check if it is less than 140. Remove the redundant checks and ensure proper validation of message length with the correct value.

>_ packages/std/sources/libs/message_decoding.move RUST

```rust
public fun safe_decode_type_2_message(message: &vector<u8>): (bool, vector<u8>,
    ↪ vector<vector<u8>>, vector<u64>) {
    // Check minimum length for LayerZero metadata (76 bytes) + fixed fields (32 + 32 + 32
        ↪ + 32 = 128 bytes)
    if (vector::length(message) < 204) {
        return (false, vector::empty(), vector::empty(), vector::empty())
    };

    // Extract base_deployer_bytes using slice (76 + 32 = 108 for start, 76 + 64 = 140 for
        ↪ end)
    if (140 > vector::length(message)) {
        return (false, vector::empty(), vector::empty(), vector::empty())
    };
    [...]
}
```

© 2025 Otter Audits LLC. All Rights Reserved.

3. `dump_unnotified_balances` returns a tuple containing the dumped amounts. However, since the returned values are exactly the same as the input parameters (`unnotified_balance_0` and `unnotified_balance_1`), it is not necessary to return anything. Moreover, in `dual_asset_base_deployer` and `single_asset_base_deployer`, `dump_unnotified_balances` does not need to check if the balances are greater than 0, as `dump_funds_to_asset_dump` already checks for the same.

>_ *deployers/dual-asset/sources/base/dual_asset_base_deployer.move* RUST

```rust
fun dump_unnotified_balances(
    [...]
    unnotified_balance_0: u64,
    unnotified_balance_1: u64
): (u64, u64) {
    let dumped_amount_0 = 0u64;
    let dumped_amount_1 = 0u64;
    if (unnotified_balance_0 > 0) {
            dump_funds_to_asset_dump(deployer_ref, global_config,
                ↪  deployer_ref.asset_0,unnotified_balance_0);
        dumped_amount_0 = unnotified_balance_0;
    };
    if (unnotified_balance_1 > 0) {
            dump_funds_to_asset_dump(deployer_ref, global_config,
                ↪  deployer_ref.asset_1,unnotified_balance_1);
        dumped_amount_1 = unnotified_balance_1;
    };
    (dumped_amount_0, dumped_amount_1)
}
```

4. In `deploy_funds_fa::test_production_deployer_deploy_none` , the first assertion in the deployer's final state verification is redundant and can be removed, as the subsequent assertion performs an equivalent check.

>_ *single-asset/tests/integration/satay/entry_functions/deploy_funds_fa.move* RUST

```rust
#[test(aptos_framework = @aptos_framework, governance = @satay, account = @DEV_ADDR)]
fun test_production_deployer_deploy_none(account: &signer) {
    satay_deployer_helpers::setup();
    [...]
    // Verify deployer's final state
    assert!(single_asset_base_deployer::last_base_balance(deployer) == 0, 0); // All base
        ↪  claimed
    assert!(primary_fungible_store::balance(deployer_addr, base_asset) == 0, 0);
    [...]
}
```

© 2025 Otter Audits LLC. All Rights Reserved.

Remediation

Remove the redundant code instances.

© 2025 Otter Audits LLC. All Rights Reserved.

Unutilized Code

OS-CPY-SUG-11

Description

1. In `process_claim`, when handing the scenario when a claim is made during the lockup period, `table::upsert` is called, which is not appropriate as if the `eth_address` already exists in `claimed_during_lockup`, `upsert` will update the existing value to true. Instead, utilize `add`, which will throw an error if the `eth_address` already exists. Moreover the assertion check ensures that reaching this point will not be possible if the `eth_address` exists in `claimed_during_lockup`.
2. In `dual_asset_base_deployer`, `claimableAmounts` structure, the `next_vest_amount` and `next_vest_timestamp` fields are not utilized.

   >_ deployers/dual-asset/sources/base/dual_asset_base_deployer.move RUST

   ```rust
   /// View struct for claimable amounts including vesting details
   struct ClaimableAmounts has copy, drop {
       [...]
       next_vest_amount: u64,
       next_vest_timestamp: u64,
       [...]
   }
   ```

3. After fixes in 5abad86, both `dump_unnotified_balances` and `dump_funds_to_asset_dump` are not utilized in `dual_asset_base_deployer` and may be removed.
4. Several artifacts remain after removing the signature-style claim. The `eip712.move` file is now unutilized, and many functions in the extensions packages are non-functional, as their referenced functions (such as `claim_relevant_assets` and `domain_separator`) no longer exist.
5. In `oapp`, `E_COMPOSE_NOT_IMPLEMENTED`, `E_INVALID_DEPLOYER_BYTES`, `E_INVALID_MESSAGE_TYPE`, `E_INVALID_MESSAGE_LENGTH`, `TRACE_CLAIM_CONFIG_SUCCESS`, `TRACE_DEPLOYER_NOT_FOUND`, `TRACE_NO_VALUE`, `TRACE_NOTIFY_FUNDS` constants and `lz_receive_impl`, `next_nonce_impl` and `get_asset_amount` functions are not utilized.
6. The codebase contains several unutilized error codes and wrong or outdated comments that obscure the code, hindering maintenance and development efficiency. Remove such dead code to streamline the codebase and ensure comments are accurate.
7. There are unutilized entries in `single_asset_base_deployer`, such as `E_SIGNATURE_EXPIRED`, `E_INVALID_SIGNATURE` or `CLAIM_TYPE_HASH` constants, `get_rs_address` function, and `std::string` and `aptos_framework::chain_id` imports.

© 2025 Otter Audits LLC. All Rights Reserved.

8. `TRACE_DEPLOYER_NOT_FOUND` and `TRACE_MISMATCHED_VECTORS` error in `notify_funds` are never going to trigger as the `OApp` will catch these cases sooner in `handle_batch_contribution_message` and `message_decoding`, returning with `TRACE_DEPLOYER_NOT_FOUND` and `TRACE_DECODE_FAILURE`, respectively.
9. The return value of `finalize_deploy_funds` is currently not utilized.

Remediation

Remove any unutilized code.

© 2025 Otter Audits LLC. All Rights Reserved.

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

© 2025 Otter Audits LLC. All Rights Reserved.

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

© 2025 Otter Audits LLC. All Rights Reserved.